

KMeans OpenMP

Mattia Baroncelli

7166468

mattia.baroncelli@edu.unifi.it

Abstract

Il progetto si concentra sull'implementazione dell'algoritmo K-Means, in particolare il confronto fra la versione sequenziale e quella parallela. K-Means è un algoritmo per l'apprendimento non supervisionato, che permette dato un insieme di punti appartenenti ad uno stesso spazio dimensionale e un valore K, di partizionare l'insieme di punti in K diversi cluster. K-Means crea i cluster assegnando ad ogni punto il centroide più vicino, andando così a minimizzare la varianza all'interno di ogni gruppo. Lo scopo principale del progetto è quindi di implementare una versione parallela dell'algoritmo tramite il framework OpenMP, valutando i vantaggi rispetto alla versione sequenziale. Si andrà inoltre a ricercare la migliore soluzione al problema della parallelizzazione.

Future Distribution Permission

The author(s) of this report give permission for this document to be distributed to Unifi-affiliated students taking future courses.

1. Introduction

L'algoritmo K-Means è un metodo che permette di formare cluster a partire da un dato insieme di punti. Questo viene fatto dando in input all'algoritmo un dataset di punti e un valore K che rappresenta il numero di cluster in cui saranno divisi i punti. L'algoritmo si basa sull'utilizzo dei centroidi, ossia K punti appartenenti al dataset, che durante l'esecuzione dell'algoritmo si andranno a spostare in base all'assegnazione dei punti del dataset ad ogni cluster. Al termine dell'algoritmo ogni punto sarà assegnato al cluster che minimizza la distanza con il proprio centroide. L'algoritmo è composto da tre fasi che vedremo nella prossima sezione ed è un algoritmo molto noto in letteratura, poichè nonostante la sua semplicità nell'implementazione risulta essere molto efficace. Nonostante questo però presenta alcune criticità, infatti all'aumentare della dimensionalità dei punti selezionati, sia per quanto riguarda il numero che per lo spazio, i tempi di esecuzione si dilatano notevolmente. Con lo scopo di superare queste limitazioni ho implementato

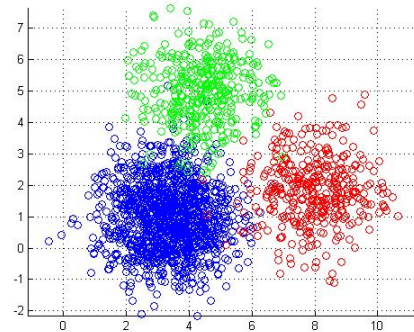


Figure 1. Esempio di clustering K-Means con evidenziate le partizioni dei punti.

la versione parallela dell'algoritmo. Attraverso il framework *OpenMP* è possibile parallelizzare parte del codice, e tramite questo andremo a valutare quanto l'impatto della parallelizzazione migliori le prestazioni dell'algoritmo.

2. Implementazione

L'algoritmo K-Means è costituito da tre fasi principali, ognuna contraddistinta da step distinti:

- Inizializzazione dei centroidi: in questa fase si andranno a selezionare casualmente K punti dal dataset che saranno utilizzati come centroidi.
- Assegnamento dei punti ai cluster: l'algoritmo andrà a cercare per ogni punto il centroide a lui più vicino e di conseguenza gli assegnerà il corrispondente cluster.
- Aggiornamento dei centroidi: una volta assegnati tutti i punti verrà fatta una media delle posizioni dei punti appartenente ad ogni cluster per aggiornare il relativo centroide.

Sull'implementazione di ogni fase ci concentreremo di seguito quando parleremo nel dettaglio del confronto fra versione sequenziale e parallela. Per quanto riguarda le condizioni di convergenza dell'algoritmo esistono varie scelte,

io ho preferito interrompere l'algoritmo quando nessun punto ha cambiato cluster dopo un'iterazione di assegnazione dei punti ai rispettivi gruppi.

2.1. OpenMP directives

Prima di iniziare con l'analisi del codice è utile vedere le direttive che andrò ad utilizzare per parallelizzare l'algoritmo nel progetto. Grazie a questo particolare tipo di istruzioni presenti in *OpenMP* è possibile creare multipli threads così da velocizzare l'esecuzione del codice, di seguito le direttive che ho usato.

- `#pragma omp parallel`: Grazie a questa istruzione è possibile generare multipli threads all'interno del programma.
- `#pragma omp for`: the *omp for* directive instructs the compiler to distribute loop iterations within the team of threads that encounters this work-sharing construct.
- `#pragma omp critical`: The *omp critical* directive identifies a section of code that must be executed by a single thread at a time.

Grazie a queste direttive non solo riusciremo a parallelizzare il codice, ma anche ad evitare problemi che potrebbero nascere proprio a causa della parallelizzazione.

2.2. Sequential version

Iniziamo l'analisi con la versione sequenziale dell'algoritmo K-Means. L'algoritmo è molto noto in letteratura e come detto prima è formato da tre parti principali. L'algoritmo è stato scritto in modo tale da potersi adattare alla versione parallela con poche aggiunte (principalmente le direttive *OpenMP*).

Listing 1. Fase di assegnamento della versione sequenziale

```
1 for (int i = 0; i < n; i++) {
2     double min_dist = std::numeric_limits<
3         double>::max();
4     int best_cluster = -1;
5
6     for (int j = 0; j < K; ++j) {
7         double dist =
8             calculateDistanceSquare(points[
9                 i].values, centroids[j]);
10        if (dist < min_dist) {
11            min_dist = dist;
12            best_cluster = j;
13        }
14    }
15    if (points[i].cluster != best_cluster) {
16        points[i].cluster = best_cluster;
17        changed = true;
18    }
19 }
```

Il primo blocco di codice che andiamo ad analizzare riguarda la fase di assegnamento di ogni punto al proprio centroide. La fase di selezione dei centroidi, infatti, non è molto interessante per l'analisi e quindi abbiamo preferito non riportarla. Nel codice si andrà ad effettuare un ciclo per ogni punto e per ogni centroide si andrà a calcolare la distanza fra quest'ultimo e il punto, andando di volta in volta ad aggiornare il parametro `min_dist` fino a trovare il centroide che minimizza la distanza con il punto selezionato. Infine se il centroide ha cambiato cluster, si va ad aggiornare il cluster e a cambiare il valore di un flag necessario per verificare la condizione di arresto. Andiamo adesso a vedere la parte del codice che riguarda il calcolo dei nuovi centroidi.

Listing 2. Fase di aggiornamento dei centroidi della versione sequenziale

```
1 std::vector<std::vector<double>>
2     new_centroids_sum(K, std::vector<double>
3         >(dimensions, 0.0));
4     std::vector<int> cluster_counts(K,
5         0);
6 for (int i = 0; i < n; ++i) {
7     int cluster_id = points[i].cluster;
8     cluster_counts[cluster_id]++;
9     for (int d = 0; d < dimensions; ++d) {
10         new_centroids_sum[cluster_id][d]
11             += points[i].values[d];
12     }
13 }
14
15 for (int j = 0; j < K; ++j) {
16     if (cluster_counts[j] > 0) {
17         for (int d = 0; d < dimensions; ++
18             d) {
19             centroids[j][d] =
20                 new_centroids_sum[j][d] /
21                 cluster_counts[j];
22         }
23     }
24 }
```

In questo blocco vengono innanzitutto inizializzati due parametri: una matrice che si occuperà di conservare le somme dei punti che appartengono ad un particolare centroide per ognuna delle sue dimensioni e un vettore che conta il numero di punti all'interno di un cluster. Si ha quindi un ciclo su ogni punto, che andrà ad incrementare il contatore del relativo cluster e andrà ad aggiornare la matrice delle somme. Una volta aggiornati questi due parametri per ogni punto si andrà ad effettuare la media dividendo i valori all'interno della matrice per il vettore delle somme di ogni centroide, ottenendo così le coordinate dei centroidi aggiornate. Senza far vedere il codice ritengo corretto citare la condizione di arresto che si verifica quando dopo un'iterazione dell'algoritmo nessun punto ha cambiato cluster rispetto all'iterazione precedente, ed è imple-

mentato con un semplice ciclo `while`.

2.3. Parallel version

Dopo aver implementato la versione sequenziale del codice vediamo adesso come è possibile passare ad una versione parallela. Le versioni parallele sviluppate in realtà sono due, che sfruttano due approcci diversi della parallelizzazione:

- Array of Structures (AoS): E' l'approccio più vicino a quanto implementato con la versione sequenziale. I dati sono organizzati in memoria in modo che il singolo punto abbia le proprie coordinate spazialmente vicine.
- Structure of Arrays (SoA): Questo approccio separa la singole componenti dei punti in array distinti, permettendo ai thread di leggere blocchi contigui di memoria durante il calcolo delle distanze.

A livello teorico quindi l'approccio SoA dovrebbe darci risultati migliori in termini di speed-up rispetto alla versione sequenziale, vedremo nella sezione degli Esperimenti se questo sarà confermato. Soffermiamoci adesso ad analizzare entrambe le versioni di codice che abbiamo implementato.

2.3.1 Array of Structures

Come detto prima questa prima versione parallela non si discosta molto da quella sequenziale, grazie all'approccio usato in precedenza è sufficiente adattare il codice prestando attenzione ad alcune problemi derivati dalla parallelizzazione. Andiamo innanzitutto a valutare la struttura della classe `Point`, che risulta essere la stessa utilizzata nella versione sequenziale

Listing 3. Implementazione dei punti nella versione AoS

```
1 struct Point {
2     int cluster;
3     std::vector<double> values;
4
5     Point(std::vector<double> v);
6 };
```

Ogni punto è quindi formato dalle proprie coordinate di appartenenza e dall'ID del proprio cluster, che è inizializzato a -1 nella fase di inizializzazione dei punti. Andiamo quindi adesso ad analizzare il codice che assegna i punti ai cluster

Listing 4. Fase di assegnamento parallela AoS

```
1 #pragma omp parallel for reduction(||:
   changed)
2 for (int i = 0; i < n; i++) {
3     double min_dist = std::numeric_limits<
       double>::max();
```

```
4     int best_cluster = -1;
5
6     for (int j = 0; j < K; ++j) {
7         double dist =
8             calculateDistanceSquare(points[
9                 i].values, centroids[j]);
10        if (dist < min_dist) {
11            min_dist = dist;
12            best_cluster = j;
13        }
14    }
15    if (points[i].cluster != best_cluster) {
16        points[i].cluster = best_cluster;
17        changed = true;
18    }
19 }
```

All'atto pratico il codice risulta essere uguale alla versione sequenziale, con l'aggiunta della direttiva `#pragma omp parallel for reduction(||:changed)`, il costrutto serve a parallelizzare il ciclo `for` sottostante che effettua il calcolo delle distanze fra punti e centroidi e assegna ogni punto a un centroide, in questo caso le variabili `min_dist` e `best_cluster` venendo dichiarate all'interno del ciclo diventano private per il thread che sta eseguendo. Lo stesso non si può dire del flag `changed` che viene inizializzato fuori dal ciclo e che può portare ad un comportamento indefinito. Se due thread cercano di scrivere nello stesso momento in quella cella di memoria possono verificarsi rallentamenti o nella peggiore delle ipotesi la corruzione del dato e di conseguenza un funzionamento errato dell'algoritmo (potrebbe fermarsi prima). Per risolvere questo problema utilizziamo il costrutto `reduction`. OpenMP andrà a creare delle copie locali della variabile `changed`. Una volta terminato il ciclo OpenMP fonde le variabili locali tramite l'operatore `OR` (che gli abbiamo passato). Il risultato pertanto sarà lo stesso ma evitando il problema citato in precedenza.

Listing 5. Fase di aggiornamento dei centroidi parallela AoS

```
1 #pragma omp parallel
2 {
3     std::vector<std::vector<double>>
4         local_centroids_sum(K, std::vector<
5             double>(dimensions, 0.0));
6     std::vector<int> local_cluster_counts(K, 0)
7     ;
8
9     #pragma omp for
10    for (int i = 0; i < n; ++i) {
11        int cluster_id = points[i].cluster;
12        local_cluster_counts[cluster_id]++;
13        for (int d = 0; d < dimensions; ++d)
14        {
15            local_centroids_sum[cluster_id][d]
16                += points[i].values[d];
17        }
18    }
19 }
```

```

12     }
13 }
14
15 #pragma omp critical
16 {
17     for (int j = 0; j < K; ++j) {
18         cluster_counts[j] +=
19             local_cluster_counts[j];
20         for (int d = 0; d < dimensions; ++
21             d) {
22             new_centroids_sum[j][d] +=
23                 local_centroids_sum[j][d];
24         }
25     }
26
27     for (int j = 0; j < K; ++j) {
28         if (cluster_counts[j] > 0) {
29             for (int d = 0; d < dimensions; ++d)
30                 {
31                     centroids[j][d] =
32                         new_centroids_sum[j][d] /
33                         cluster_counts[j];
34                 }
35         }
36     }
37 }

```

La fase di aggiornamento dei centroidi si apre con, oltre alla dichiarazione degli accumulatori di somme e numero di punti, anche con quella di accumulatori locali per ogni cluster, che assegneremo ad ogni thread. Anche stavolta tramite la direttiva `#pragma omp for`, andremo a parallelizzare il ciclo. In maniera analoga a prima ogni thread andrà ad aggiornare i propri accumulatori. Una volta terminato l'aggiornamento degli accumulatori per ogni punto, è necessario aggiornare gli accumulatori globali, andandogli ad assegnare la somma dei rispettivi contatori parziali. Questa operazione può portare al verificarsi di una race condition, dove due threads leggono un valore e provano a sovrascrivere gli accumulatori globali con i propri valori, rischiando di portare a risultati errati. Per risolvere questo problema abbiamo adottato il costrutto `#pragma omp critical` che assicura che un solo thread alla volta aggiorni i contatori globali.

2.3.2 Structure of Arrays

Andiamo a vedere in questa sezione come è possibile implementare l'algoritmo con il paradigma SoA. Di seguito il codice che illustra la struttura l'implementazione dei punti con questa tecnica.

Listing 6. struct Implementazione dei punti nella versione SoA

```

1 struct DatasetSoA {
2     int num_points;

```

```

3     int dimensions;
4
5     std::vector<double> coords;
6     std::vector<int> clusters;
7 };

```

Il dataset viene implementato tramite la struct `DatasetSoA` che ha come attributi, il numero totale di punti e di dimensioni, il vettore che rappresenta le coordinate dei punti e un array separato per etichette dei clusters. La funzione `convertToSoA` si occuperà di convertire i punti passati nel formato AoS, in questo formato, tramite un ciclo `for`.

Listing 7. ciclo `for` all'interno di `convertToSoA`

```

1 for (int i = 0; i < soa.num_points; ++i) {
2     soa.clusters[i] = aos_dataset[i].cluster
3     ;
4     for (int d = 0; d < soa.dimensions; ++d)
5     {
6         // Indice piatto: (riga *
7         // numero_colonne) + colonna
8         soa.coords[i * soa.dimensions + d] =
9             aos_dataset[i].values[d];
10    }
11 }

```

In particolare la funzione prende in input il dataset in formato AoS e innanzitutto assegna il medesimo cluster di ogni punto alla rappresentazione in formato SoA, successivamente attraverso la tecnica del flat index, costruisce l'array di coordinate. In questo modo è possibile passare facilmente dalla rappresentazione AoS a SoA. Anche l'algoritmo avrà necessità di essere adattato, in particolare, oltre a una diversa versione dell'algoritmo per calcolare la distanza, che sfrutta anch'esso il flat index; abbiamo una differenza sostanziale nella terza fase dell'algoritmo.

Listing 8. Fase di aggiornamento dei centroidi SoA

```

1 std::vector<double> new_centroids_sum(K *
2     dims, 0.0);
3 std::vector<int> cluster_counts(K, 0);
4
5 #pragma omp parallel
6 {
7     std::vector<double> local_centroids_sum(K *
8         dims, 0.0);
9     std::vector<int> local_cluster_counts(K, 0)
10     ;
11
12     for (int i = 0; i < n; ++i) {
13         int cluster_id = dataset.clusters[i];
14         local_cluster_counts[cluster_id]++;
15
16         for (int d = 0; d < dims; ++d) {
17             local_centroids_sum[cluster_id * dims
18                 + d] += dataset.coords[i * dims +
19                 d];
20         }
21     }
22 }

```

```

16 }
17
18 {
19     for (int j = 0; j < K; ++j) {
20         cluster_counts[j] +=
21             local_cluster_counts[j];
22         for (int d = 0; d < dims; ++d) {
23             new_centroids_sum[j * dims + d] +=
24                 local_centroids_sum[j * dims +
25                     d];
26         }
27     }
28 }
29
30 for (int j = 0; j < K; ++j) {
31     if (cluster_counts[j] > 0) {
32         for (int d = 0; d < dims; ++d) {
33             centroids[j * dims + d] =
34                 new_centroids_sum[j * dims + d] /
35                 cluster_counts[j];
36         }
37     }
38 }

```

Nelle operazioni compiute il codice è molto simile a quello scritto per la versione AoS. Fin da subito però è possibile notare come sia gli accumulatori globali che quelli locali siano cambiate. In questa versione infatti si richiede per le somme dei centroidi una allocazione per un blocco continuo, al contrario delle metriche allocate in precedenza. Inoltre il costrutto di parallelizzazione è diventato `#pragma omp for schedule(static)`, che forza l'assegnazione dei blocchi contigui ad uno stesso thread (cosa che è stata aggiunta anche alla fase di assegnamento dei punti). Infine sia il calcolo della somma dei centroidi e della media finale vengono fatte utilizzando la rappresentazione piatta.

3. Esperimenti

Iniziamo adesso la sezione del report che riguarda gli esperimenti effettuati, abbiamo visto come i tre diversi tipi di implementazione variano a livello di codice, adesso vediamo quanto variano a livello di prestazioni. Per farlo è necessario partire innanzitutto dalla creazione del dataset.

3.1. Dataset

I dataset sono stati creati attraverso una funzione Python creata appositamente `generate_dataset`. La funzione prende in input:

- Numero di punti (N)
- Numero di cluster (K)
- Dimensione del piano su cui giacciono i punti

- Parametro di dispersione dei punti

e restituisce in output un dataset in formato csv, che contiene sulle righe le coordinate di ogni punto. Poiché k-means è un algoritmo non supervisionato nel dataset non è presente il cluster di appartenenza, nonostante ciò il numero di cluster è un parametro in input per creare dataset che abbiamo punti che vadano a formare cluster definiti. Per convertire i punti in formato SoA verrà poi usata la funzione apposita definita nella sezione precedente; la conversione come era logico aspettarsi non concorrerà alla misurazione del tempo di esecuzione dell'algoritmo.

3.2. Aumento N AoS

Iniziamo quindi gli esperimenti valutando le prestazioni della tecnica Structure of Arrays su vari dataset. Il primo esperimento verrà fatto aumentando la dimensione del numero di punti (N), mantenendo fermo il numero di cluster K a 10. Andremo inoltre ad aumentare gradualmente il numero threads in esecuzione, per verificare se c'è un miglioramento delle performance all'aumentare dei threads.

3.3. Aumento K

4. Conclusioni

4.1. The ruler

4.2. Mathematics

Please number all of your sections and displayed equations. It is important for readers to be able to refer to any particular equation. Just because you didn't refer to it in the text doesn't mean some future reader might not need to refer to it. It is cumbersome to have to use circumlocutions like "the equation second from the top of page 3 column 1". (Note that the ruler will not be present in the final copy, so is not an alternative to equation numbers). All authors will benefit from reading Mermin's description of how to write mathematics: <http://www.cvpr.org/doc/mermin.pdf>.

4.3. Miscellaneous

Compare the following:

$$\begin{array}{ll} \$conf_a\$ & conf_a \\ \$\mathit{conf}_a\$ & conf_a \end{array}$$

See The T_EXbook, p165.

The space after *e.g.*, meaning "for example", should not be a sentence-ending space. So *e.g.* is correct, *e.g.* is not. The provided `\eg` macro takes care of this.

When citing a multi-author paper, you may save space by using "et alia", shortened to "*et al.*" (not "*et. al.*" as "*et*" is a complete word.) However, use it only when there are three or more authors. Thus, the following is correct: "Frobination has been trendy lately. It was introduced

by Alpher [3], and subsequently developed by Alpher and Fotheringham-Smythe [1], and Alpher *et al.* [2].”

This is incorrect: “... subsequently developed by Alpher *et al.* [1] ...” because reference [1] has just two authors. If you use the `\etal` macro provided, then you need not worry about double periods when used at the end of a sentence as in Alpher *et al.*

For this citation style, keep multiple citations in numerical (not chronological) order, so prefer [1, 3, 4] to [3, 1, 4].

5. Formatting your paper

All text must be in a two-column format. The total allowable width of the text area is $6\frac{7}{8}$ inches (17.5 cm) wide by $8\frac{7}{8}$ inches (22.54 cm) high. Columns are to be $3\frac{1}{4}$ inches (8.25 cm) wide, with a $\frac{5}{16}$ inch (0.8 cm) space between them. The main title (on the first page) should begin 1.0 inch (2.54 cm) from the top edge of the page. The second and following pages should begin 1.0 inch (2.54 cm) from the top edge. On all pages, the bottom margin should be 1-1/8 inches (2.86 cm) from the bottom edge of the page for 8.5×11 -inch paper; for A4 paper, approximately 1-5/8 inches (4.13 cm) from the bottom edge of the page.

5.1. Margins and page numbering

All printed material, including text, illustrations, and charts, must be kept within a print area 6-7/8 inches (17.5 cm) wide by 8-7/8 inches (22.54 cm) high.

5.2. Type-style and fonts

Wherever Times is specified, Times Roman may also be used. If neither is available on your word processor, please use the font closest in appearance to Times to which you have access.

MAIN TITLE. Center the title 1-3/8 inches (3.49 cm) from the top edge of the first page. The title should be in Times 14-point, boldface type. Capitalize the first letter of nouns, pronouns, verbs, adjectives, and adverbs; do not capitalize articles, coordinate conjunctions, or prepositions (unless the title begins with such a word). Leave two blank lines after the title.

AUTHOR NAME(s) and AFFILIATION(s) are to be centered beneath the title and printed in Times 12-point, non-boldface type. This information is to be followed by two blank lines.

The **ABSTRACT** and **MAIN TEXT** are to be in a two-column format.

MAIN TEXT. Type main text in 10-point Times, single-spaced. Do NOT use double-spacing. All paragraphs should be indented 1 pica (approx. 1/6 inch or 0.422 cm). Make sure your text is fully justified—that is, flush left and flush right. Please do not place any additional blank lines between paragraphs.

Method	Frobnability
Theirs	Frumpy
Yours	Frobbly
Ours	Makes one's heart Frob

Table 1. Results. Ours is better.

Figure and table captions should be 9-point Roman type as in Figures ?? and 2. Short captions should be centred.

Callouts should be 9-point Helvetica, non-boldface type. Initially capitalize only the first word of section titles and first-, second-, and third-order headings.

FIRST-ORDER HEADINGS. (For example, **1. Introduction**) should be Times 12-point boldface, initially capitalized, flush left, with one blank line before, and one blank line after.

SECOND-ORDER HEADINGS. (For example, **1.1. Database elements**) should be Times 11-point boldface, initially capitalized, flush left, with one blank line before, and one after. If you require a third-order heading (we discourage it), use 10-point Times, boldface, initially capitalized, flush left, preceded by one blank line, followed by a period and your text on the same line.

5.3. Footnotes

Please use footnotes¹ sparingly. Indeed, try to avoid footnotes altogether and include necessary peripheral observations in the text (within parentheses, if you prefer, as in this sentence). If you wish to use a footnote, place it at the bottom of the column on the page on which it is referenced. Use Times 8-point type, single-spaced.

5.4. References

List and number all bibliographical references in 9-point Times, single-spaced, at the end of your paper. When referenced in the text, enclose the citation number in square brackets, for example [4]. Where appropriate, include the name(s) of editors of referenced books.

5.5. Illustrations, graphs, and photographs

All graphics should be centered. Please ensure that any point you wish to make is resolvable in a printed copy of the paper. Resize fonts in figures to match the font in the body text, and choose line widths which render effectively in print. Many readers (and reviewers), even of an electronic copy, will choose to print your paper in order to read it. You cannot insist that they do otherwise, and therefore must not assume that they can zoom in to see tiny details on a graphic.

¹This is what a footnote looks like. It often distracts the reader from the main flow of the argument.

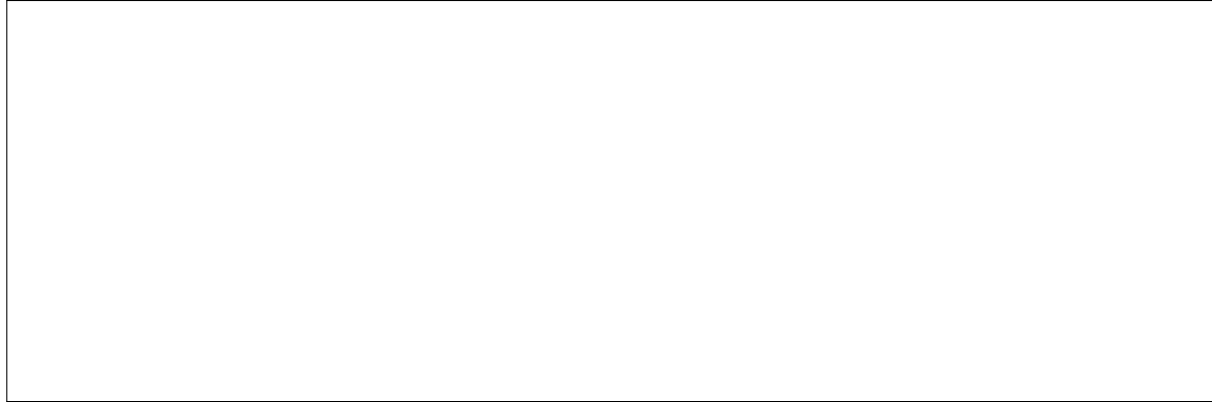


Figure 2. Example of a short caption, which should be centered.

When placing figures in $\text{\LaTeX}$, it's almost always best to use `\includegraphics`, and to specify the figure width as a multiple of the line width as in the example below

```
\usepackage[dvips]{graphicx} ...  
\includegraphics[width=0.8\linewidth]  
    {myfile.eps}
```

5.6. Color

Color is valuable, and will be visible to readers of the electronic copy. However ensure that, when printed on a monochrome printer, no important information is lost by the conversion to grayscale.

References

- [1] A. Alpher, , and J. P. N. Fotheringham-Smythe. Frobnication revisited. *Journal of Foo*, 13(1):234–778, 2003.
- [2] A. Alpher, , J. P. N. Fotheringham-Smythe, and G. Gamow. Can a machine frobnicate? *Journal of Foo*, 14(1):234–778, 2004.
- [3] A. Alpher. Frobnication. *Journal of Foo*, 12(1):234–778, 2002.
- [4] Authors. The frobnicatable foo filter, 2006. ECCV06 submission ID 324. Supplied as additional material `eccv06.pdf`.

6. Appendix

If your course project is part of a larger project from another class or research lab, please fill in this section and clearly spell out the following items:

1. Explicitly explain what the computer vision components are in this course project;
2. Explicitly list out all of your own contributions in this project in terms of:
 - (a) ideas
 - (b) formulations of algorithms

- (c) software and coding
- (d) designs of experiments
- (e) analysis of experiments

3. Verify and confirm that you (and your partner currently taking CS231A) are the sole author(s) of the writeup. Please provide papers, theses, or other documents related to this project so that we can compare with your own writeup.